

Faz 3 Ar-Ge ve Ticarileşme Süreçleri: İleri Teknoloji, Açık Bankacılık ve Çevrimdışı Öncelikli Senkronizasyon Mimarisi

Faz 3 Ar-Ge Ekosisteminin Ontolojisi ve Kavramsal Çerçevesi

İleri teknoloji projelerinin yaşam döngüsünde Faz 3, kavram kanıtlama (Proof of Concept) ve sınırlı prototipleme (Faz 1 ve Faz 2) aşamalarının laboratuvar ortamındaki başarılarının ardından gelen, ürünün endüstriyel ölçekte ticarileşmeye, küresel pazar dinamiklerine entegrasyona ve geniş kullanıcı kitlelerine ulaşmaya hazırlandığı en kritik geliştirme safhasıdır. Bu aşama, salt bir kod yazım veya donanım montaj süreci olmaktan çıkarak; yasal uyumluluk, yüksek ölçeklenebilirlik, veri tutarlılığı ve finansal sürdürülebilirlik gibi çok katmanlı disiplinlerin kusursuz bir orkestrasyonunu gerektirir. Faz 3'ün temel karakteristiklerini ve barındırdığı riskleri tam anlamıyla kavrayabilmek için, bu kavramın kökenlerinin dayandığı ve en katı bilimsel metotlarla uygulandığı klinik araştırma ekosistemini incelemek, yazılım mühendisliği için emsalsiz içgörüler sunmaktadır.

İlaç ve tıbbi cihaz geliştirme süreçlerinde süreçler, Pre-Klinik geliştirme aşamasında laboratuvar ortamında yapay zeka (AI) ve hücre testleri ile başlar ve potansiyel riskler bertaraf edildikten sonra insanlı deneylere geçiş için hükümet düzenleyici kurumlarından onay alınır. Faz 1, temel güvenliği ve dozajı test ederken; Faz 2, ürünün etkinliğini (efficacy) ve potansiyel yan etkilerini 100 ila 300 kişilik sınırlı gruplar üzerinde değerlendirir. Faz 3 ise, etkinliğin kesin olarak doğrulandığı, yeni ürünün pazardaki mevcut standart teknolojilerle (standard treatments) karşılaştırıldığı ve 300 ila 3000 (hatta COVID-19 aşılarında olduğu gibi 40.000) katılımcıyla gerçekleştirilen en büyük ölçekli stres testidir. Bu aşama genellikle dört yıla kadar sürebilmektedir.

Bu analojiyi bir yazılım Ar-Ge projesinin Faz 3'üne uyarladığımızda, Faz 2'den Faz 3'e geçişin barındırdığı devasa riskler açıkça ortaya çıkmaktadır. Araştırmalar, Faz 2 aşamasını yüzeysel bir hızla geçip doğrudan Faz 3'e atlayan projelerin, Faz 3'e daha hızlı ulaşsalar da ilerleyen aşamalarda çok daha yüksek başarısızlık oranlarıyla karşılaştığını göstermektedir. Amerikan Gıda ve İlaç Dairesi (FDA) tarafından incelenen ve Faz 2 ile Faz 3 sonuçları arasında tutarsızlık bulunan 22 vakanın 14'ünde, Faz 3 çalışmaları Faz 2'deki etkinlik bulgularını doğrulayamamıştır. Yazılım mimarisinde bu durum, Beta aşamasında (Faz 2) 100 test kullanıcısı ile sorunsuz çalışan bir veritabanı senkronizasyon yapısının veya bulut mimarisinin, Faz 3 ticarileşme aşamasında 10.000 eşzamanlı kullanıcıya ulaştığında kilitlenmesi, ölümcül veri tutarsızlıkları (conflict) yaratması veya devasa bellek sızıntılarına (memory leak) yol açması olarak tezahür eder.

Bunun yanı sıra Faz 3, maliyetlerin logaritmik olarak tırmandığı ve yatırım geri dönüş (ROI) hesaplamalarının acımasızlaştığı bir dönemdir. Klinik araştırmalarda, örneğin bir tüberküloz aşısının sadece Faz 3 denemesi 500 milyon doların üzerinde maliyet yaratabilmektedir. Yazılım

Faz 3'ünde de dağıtık sunucu mimarileri, veri tabanı replikasyonları, sızma testleri (penetration testing) ve yasal regülasyon uyum süreçleri benzer bir bütçe yutucu karaktere bürünür. Tüm projeler ele alındığında, Faz 3 sonrası başarı oranının yalnızca %20-25 seviyelerinde seyrettiği görülmektedir. Bu amansız istatistik, ticarileşme ve fikri mülkiyet (IP) ekipleri ile Ar-Ge yöneticilerinin, projenin düzenleyici onay yolları ve patent/telif hakları koruma sürelerini dikkate alarak ürün yaşam döngüsünü uzatma stratejilerini henüz Faz 3 başlangıcında geliştirmelerini zorunlu kılar. Bu bağlamda, teknoloji portföy yöneticilerinin ticari başarıyı doğru fiyatlayabilmeleri için klinik, fikri mülkiyet ve ticari boyutları entegre eden "Gerçek Efektif Patent Ömrü" (True Effective Patent Life) veya yazılım dünyasındaki karşılığı olan "Pazara Giriş ve Teknoloji Eskime Süresi" formülasyonlarını Faz 3'ün ilk adımlarında tamamlamaları elzemdir.

Ar-Ge Aşaması	Klinik Araştırma Metodolojisi Karşılığı	İleri Yazılım ve Finansal Teknoloji Karşılığı	Temel Risk Faktörleri ve Başarısızlık Nedenleri
Faz 1	Güvenlik ve Dozaj Kontrolü (10-50 kişi)	Çekirdek Mimari Kurulumu, POC ve Temel Güvenlik	Altyapısal mimari hatalar, konseptin teknik olarak imkansız olması.
Faz 2	Etkinlik ve Yan Etkiler (100-300 kişi)	Sınırlı Kullanıcı Testi, MVP Doğrulaması (Beta)	Beklenen performans hedeflerine ulaşılabilmesi, kullanıcı benimseme eksikliği.
Faz 3	Kesin Efficacy, Standartlarla Karşılaştırma, Yüksek Ölçek (300-40.000 kişi)	Yüksek Ölçeklendirme (Load Testing), Dağıtık Senkronizasyon, Regülasyon Uyumlu	Faz 2'deki başarının yüksek yük altında doğrulanabilmesi (14/22 FDA kuralı), sistemin çökmesi, finansal kaynakların tükenmesi.

Türkiye Ekosisteminde Faz 3 Finansal Modeli: TEYDEB Destekleri ve 4691 Sayılı Kanun

Faz 3 projelerinin gerektirdiği devasa bütçelerin sürdürülebilir bir biçimde finanse edilebilmesi için, geliştirme faaliyetlerinin Türkiye Cumhuriyetinin sağlamış olduğu güçlü yasal ve finansal teşvik ekosistemine entegre edilmesi stratejik bir zorunluluktur. Türkiye bağlamında bu süreçler, öncelikli olarak TÜBİTAK Teknoloji ve Yenilik Destek Programları Başkanlığı (TEYDEB) mekanizmaları ve 4691 sayılı Teknoloji Geliştirme Bölgeleri Kanunu çerçevesinde yönetilmektedir.

TÜBİTAK TEYDEB Faz 3 ticarileşme ve Ar-Ge destekleri kapsamında, uluslararası programların (örneğin EUREKA Network, EUROSARS ve EUREKA Küme/Şemsiye gibi doğrudan çok uluslu ortaklı projelerin) özel şartlarına tabi olan dinamik başvuru dönemleri işletilmektedir. Çağrılı uluslararası programlarda belirli başvuru süreleri bulunurken, sürekli başvuru alınan uluslararası programlarda zaman kısıtlaması bulunmamaktadır. Bu programlar çerçevesinde, geliştirilecek Faz 3 projesinin KOBİ ölçeğinde bir işletme tarafından yürütülmesi halinde %75 oranında, büyük ölçekli bir şirket tarafından yürütülmesi halinde ise %60 oranında hibe desteği sağlanabilmektedir. Proje yönetimini finansal açıdan en çok rahatlatan unsurlardan biri ise, proje

bütçesinin %25'ine kadar ön ödeme (avans) imkanının sunulmasıdır. Bu peşin sermaye enjeksiyonu, Faz 3'ün gerektirdiği yüksek kapasiteli sunucu tahsisleri, uluslararası lisanslamalar ve nitelikli iş gücü istihdamının ilk safhalarında adeta bir can suyu görevi görür.

Bir Faz 3 projesinin TEYDEB standartlarında kabul görebilmesi için başvuru dokümantasyonunun son derece analitik ve hedefe yönelik hazırlanması gerekir. Proje ön bilgilerinde projenin adı, bilimsel ve teknolojik alanları, Ar-Ge çalışmalarının yürütüleceği ve proje çıktılarının kullanılacağı sektörler net bir şekilde tanımlanmalıdır. Ticarileşme aşamasının temel göstergesi olan ürünün ekonomik faaliyet sınıfını belirten NACE kodu ile projenin ihracat potansiyelini doğrudan ilgilendiren Gümrük Tarife İstatistik Pozisyonu (GTİP) kodunun tespiti zorunludur. Bununla birlikte projenin yenilikçi yönleri, kullanılacak teknik ve teknolojiler ile ekonomik ve ulusal kazanımların da başvuru aşamasında detaylandırılması; projenin başlangıç Teknoloji Hazırlık Seviyesi (TRL) ile proje sonunda ulaşılması hedeflenen TRL'nin kesin hatlarla çizilmesi gerekmektedir.

Projenin hukuki operasyon merkezi ise 4691 sayılı Teknoloji Geliştirme Bölgeleri (Teknopark) Kanunu ile güvence altına alınmaktadır. 16 Şubat 2016 tarihli ve 6676 sayılı Kanun ile yapılan değişiklikler ve sonrasındaki mevzuat güncellemeleri uyarınca, yönetici şirketlerin bu kanun uygulaması kapsamında elde ettikleri kazançlar ile bölgede faaliyet gösteren gelir ve kurumlar vergisi mükelleflerinin münhasıran bu bölgedeki yazılım, tasarım ve Ar-Ge faaliyetlerinden elde ettikleri kazançları 31 Aralık 2028 tarihine kadar gelir ve kurumlar vergisinden tamamen müstesnadır. Faz 3 aşamasındaki bir projenin ticarileşerek ilk lisans veya abonelik gelirlerini elde etmeye başladığı an, bu vergi istisnası sayesinde yaratılan serbest nakit akışı doğrudan Ar-Ge döngüsüne yeniden yatırılarak teknolojik eskimenin önüne geçilmesini sağlar.

Teknopark teşvikleri yalnızca gelir vergisiyle sınırlı kalmayıp, operasyonel giderleri de drastik ölçüde düşüren mekanizmalara sahiptir. 4691 sayılı Kanun kapsamında yürütülen yazılım, Ar-Ge, yenilik ve tasarım projeleri ile ilgili araştırmalarda kullanılmak üzere ithal edilen her türlü eşya gümrük vergisinden ve ilgili fonlardan; bu kapsamda düzenlenen kağıtlar ve yapılan işlemler ise damga vergisi ve harçlardan müstesnadır. Gelişen teknolojiler için donanım prototiplerinin veya özel sunucu çiplerinin ithalatının vergisiz yapılabilmesi büyük bir maliyet avantajıdır. İnsan kaynağı tarafında ise, 2021 yılında 7263 sayılı Kanun ile eklenen fıkra uyarınca stajyer istihdam eden firmalar, Cumhurbaşkanlığı Strateji ve Bütçe Başkanlığı ile Hazine ve Maliye Bakanlığının yönetmelikleri çerçevesinde doğrudan desteklenebilmektedir. Küresel ölçekte rekabet edebilecek bir Faz 3 projesi için yetenek havuzunu genişletmek amacıyla, 4875 sayılı Doğrudan Yabancı Yatırımlar Kanunu ve 4817 sayılı Yabancıların Çalışma İzinleri Hakkında Kanun hükümleri çerçevesinde yabancı uyruklu yönetici ve vasıflı Ar-Ge personeli çalıştırılabilmesi sağlanmıştır. Hatta yabancı özel hukuk tüzel kişilerinin bölge yönetici şirketlerine iştirak edebilmesi ve Ar-Ge faaliyetlerini bölgede yürütebilmesi, uluslararası yatırım (Venture Capital) entegrasyonu için hukuki zemini hazır hale getirmektedir.

Finansal Teknolojilerde Faz 3 Entegrasyonu: Açık Bankacılık ve ÖHVPS 2.0 Mimarisi

Geliştirilecek Faz 3 Ar-Ge projesinin, kullanıcıların finansal varlıklarıyla veya ödeme süreçleriyle en ufak bir teması bulunuyorsa, Türkiye'deki regülatif finansal teknolojiler altyapısına mutlak surette tam entegre edilmesi şarttır. Türkiye Cumhuriyet Merkez Bankası (TCMB) tarafından açık bankacılık altyapısında başlatılan yeni faz doğrultusunda, Bankalararası Kart Merkezi (BKM) tarafından geliştirilen Ödeme Hizmetleri Veri Paylaşım Servisleri'nin (ÖHVPS) 2.0.0 sürümü devreye alınmıştır. Bu sistem güncellenmeden önce sadece vadesiz hesap bakiyelerinin

ve basit hesap hareketlerinin görüntülenmesine olanak sağlarken, yeni sürümle birlikte açık bankacılık hizmetlerinin kapsamı devasa bir ölçekte genişletilmiştir.

Yeni mimaride yalnızca hesap verileri değil, kredi kartı bilgileri ve kart hareketleri de BKM GEÇİT API'leri ve açık bankacılık altyapısı üzerinden anlık olarak erişilebilir hale gelmiştir. Böylece kullanıcıların finansal verileri çok daha bütüncül ve 360 derecelik bir yapıda görüntülenebilirken, geliştirici fintech şirketleri ve Faz 3 projeleri için de yapay zeka (AI) tabanlı kredi skorlama, akıllı bütçe yönetimi (PFM) ve tüketici davranışı analizi yapabilmelerine olanak tanıyan emsalsiz ve zengin bir veri seti oluşmuştur.

Bu entegrasyonun Faz 3 projesine kazandıracığı en stratejik ticarileşme yeteneği ise ödeme tarafındaki güncellemelerde yatmaktadır. ÖHVPS 2.0 sistemine eklenen ileri tarihli ödeme emri ve düzenli ödeme emri (recurring payment) başlatma özellikleri, planlı ve tekrarlayan ödeme senaryolarını standart hale getirmiştir. Yazılım dünyasında ticarileşmenin ve sürdürülebilir gelir modelinin omurgasını oluşturan abonelik bazlı (SaaS) iş modelleri ve düzenli tahsilat yapan e-ticaret platformları için bu gelişme, aracı kurumlara ödenen komisyonları sıfırlayan veya minimize eden yeni kullanım alanları yaratmaktadır. Hukuki dayanağını 6493 sayılı Kanunun 12. maddesinin birinci fıkrasının (f) bendinden alan "Ödeme Emri Başlatma Hizmeti", ödeme hizmeti kullanıcısının isteği üzerine başka bir hizmet sağlayıcıda bulunan hesaptan doğrudan transfer başlatılmasını yasal güvenceye almaktadır.

TCMB verilerine göre açık bankacılık altyapısı bugün itibarıyla 16,4 milyon aktif kullanıcıya ve günlük ortalama 12,3 milyon işlem hacmine ulaşmış durumdadır. Sisteme entegre olan 53 finansal katılımcı ile birlikte bu devasa veri otoyolu, Faz 3 aşamasındaki projenin karşılayacağı trafik yükünün boyutlarını da belirlemektedir. Bu derece yüksek bir throughput (işlem hacmi) mimarisinin sahada etkin şekilde hayata geçirilebilmesi için, API yönetimi, MTLS (Mutual TLS) tabanlı uçtan uca güvenlik, JWS (JSON Web Signature) şifreleme, dinamik sertifikasyon ve anlık izleme (monitoring) katmanlarını tek çatı altında toplayan OBA Suite Teknoloji Platformu benzeri profesyonel çözüm mimarileri kritik rol oynar. Bu tür ara katman platformları, Faz 3 yazılımlarının BKM ve TCMB gereksinimlerine saniyelik bazda uyumunu hızlandırırken, açık bankacılık servislerinin çökmeden, ölçeklenebilir ve sürdürülebilir biçimde sunulmasını mümkün kılmaktadır.

ÖHVPS 2.0 (Açık Bankacılık) Özelliği	Faz 3 Projesi İçin Ticari / Teknik Çıktısı	Kullanım Senaryosu
Kart Bilgileri ve Hareketleri Erişimi	Gelişmiş veri seti üzerinden AI tabanlı finansal analiz yeteneği.	Kullanıcının harcama alışkanlıklarına göre dinamik bütçe optimizasyonu.
İleri Tarihli ve Düzenli Ödeme Emri	Abonelik (SaaS) gelir modellerinde tahsilatın doğrudan banka üzerinden otomleştirilmesi.	Kurumsal lisans yenilemelerinin her ayın 1'inde otomatik çekilmesi.
Ödeme Emri Başlatma (6493/12-f)	Aracı POS/Sanal POS komisyonlarının bypass edilerek doğrudan hesaptan hesaba transfer.	B2B platformlarında yüksek hacimli fatura ödemelerinin komisyonsuz tahsilatı.
Günlük 12.3 Milyon İşlem Hacmi	Faz 3 projesinin yük testlerinde (load testing) saniyelik istek sayısının referans alınması.	Yüksek ölçeklenebilir, dağıtık ve asenkron kuyruk mimarisinin zorunlu inşası.

Dağıtık Sistemler ve Faz 3 Mimari Tasarımı: Çevrimdışı

Öncelikli (Offline-First) Yaklaşım

Günümüzün hiper-bağlantılı dünyasında dahi, saha operasyonları yürüten sağlık çalışanları, lojistik ekipleri veya seyahat halindeki kurumsal kullanıcılar için ağ bağlantısı (network connectivity) her zaman garanti edilemez. Faz 3 seviyesindeki bir ticarileşme projesinin ürün kalitesini belirleyen en keskin sınır, bağlantının koptuğu anlarda sistemin nasıl tepki verdiğiidir. Tüm bu ihtiyaçlar, geleneksel "Bulut Öncelikli" veya "Yalnızca Çevrimiçi" mimarilerin yerini, karmaşık ancak son derece güvenilir olan "Çevrimdışı Öncelikli" (Offline-First) mimarilerin almasına neden olmuştur.

Çevrimdışı öncelikli mimarinin merkezinde, yerel uygulama verileri ile arka uç (backend) sunucu verilerini kusursuz ve anlık bir şekilde senkronize tutan güçlü bir eşzamanlama katmanı yatar. Geleneksel sistemlerde, uygulamalar veriyi göstermek için geçici bellek (caching) yöntemlerini kullanır. Ancak bir kullanıcının cihazındaki önbellek genellikle yaklaşık bir aylık yoğun kullanım sonrasında dolar ve sistem performansını korumak için eski veriler temizlenir. Eğer kullanıcı çevrimdışı iken eski bir veriye (örneğin bir ay önceki finansal rapora veya hasta kayıtlarına) ulaşmak isterse, veri temizlendiği için sistem hizmet veremez. Buna karşın çevrimdışı öncelikli mimari, "Veri Kapsamlandırma" (Data Scoping) veya "Sync Streams" özellikleri sayesinde, geliştiricilerin hangi verilerin kullanıcının cihazındaki yerel veritabanında süresiz (indefinitely) olarak saklanacağını kesin kurallarla tanımlamasına olanak tanır. Yerel veritabanları (özellikle SQLite tabanlı yapılar), önbelleklere kıyasla hem çok daha fazla veriyi depolayabilir hem de karmaşık SQL sorguları çalıştırırken performanstan taviz vermez.

Bu vizyon doğrultusunda Faz 3 projesinin mimari altyapısı, istemci tarafında Flutter tabanlı çapraz platform (cross-platform) yetenekleriyle, yerel veritabanı olarak Drift (SQLite ORM) ve arka uç olarak güçlü bir PostgreSQL çözümü olan Supabase etrafında şekillendirilmelidir. Bu uçları birbirine bağlayacak asıl senkronizasyon motoru ise PowerSync olacaktır. PowerSync, uygulamanın ağı olan bağımlılığını ortadan kaldırarak temel bulut bağlantılı uygulamaları, her koşulda sorunsuz çalışan gerçek çevrimdışı öncelikli deneyimlere dönüştürür. İnternet bağlantısı kesildiğinde yapılan tüm değişiklikler (CRUD işlemleri) önce yerel SQLite veritabanına yazılır, PowerSync bu işlemleri bir yükleme kuyruğunda (upload queue) bekletir ve cihaz tekrar çevrimiçi olduğunda bu kuyruk otomatik olarak arka uca boşaltılır.

Bu yapının arkasında yatan teorik tutarlılık modeli, dağıtık sistemler biliminde "Nedensel+ Tutarlılık" (Causal+ Consistency) olarak adlandırılan ve bu tür mimariler için mümkün olan en güçlü tutarlılık garantisidir. Sunucu mutabakatı (server reconciliation) ile desteklenen bu otomatik eşzamanlama, tam yerel veritabanını kullanıcının çevrimdışı olduğu son ana kadar sunucuyla tutarlı tutar. Örneğin asistanı tarafından hasta tablosunda değişiklik yapılan bir doktor, cihazı internete bağlanıp güncellemeyi aldığı andan itibaren o hastanın dosyasına girmemiş olsa dahi, arka planda tüm veritabanı nedensel sıralamaya uygun olarak senkronize edilir. PowerSync'in sağladığı bu altyapı, çevrimdışı veri depolamayı yönetme, değişiklikleri izleme ve kuyruğa alma, bağlantı geri geldiğinde eşzamanlama sağlama ve çakışmaları çözme gibi aksi takdirde sıfırdan geliştirilmesi son derece hata eğilimli (error-prone) ve zaman alıcı olan tüm yükleri üstlenir. Açık kaynak ekosisteminde de bu mimariye benzer spesifik yığınlar (stack) için geliştirilmiş, trafiği azaltan ve gerçek zamanlı abonelik sayısını optimize eden "syncable" gibi Dart tabanlı kütüphaneler de küçük çaplı alternatifler olarak varlığını sürdürmektedir.

Kısmi Senkronizasyon (Partial Sync) ve Veri

Kapsamlandırma Algoritmaları

Veri güvenliği, Kişisel Verilerin Korunması Kanunu (KVKK) uyumluluğu ve mobil cihazların kısıtlı depolama kapasiteleri göz önüne alındığında, devasa bir arka uç veritabanının tamamının (örneğin milyonlarca işlemin yer aldığı açık bankacılık veri setlerinin) tek bir kullanıcının cihazına senkronize edilmesi hem teknik bir intihar hem de hukuki bir ihlaldir. Faz 3 seviyesindeki bir uygulamada "Kısmi Senkronizasyon" (Partial Sync), yani sistemin yalnızca her bir kullanıcı için spesifik bir veri alt kümesini (subset) cihazla eşitlemesi zorunludur.

PowerSync ekosisteminde bu veri alt kümeleri, "Sync Streams" (veya önceki adıyla Sync Rules / Senkronizasyon Kuralları) adı verilen YAML tabanlı konfigürasyon dosyalarıyla yönetilir. Her bir PowerSync hizmet örneği (instance), üzerinde dağıtılmış bir senkronizasyon kural setine sahiptir. Bu YAML dosyasında geliştiriciler, veri kümelerini bucket_definitions (kova tanımlamaları) altında SQL benzeri sorgularla belirlerler.

Sistemin kalbinde, dinamik parametreleri alarak hangi kovaların oluşturulacağına karar veren parametre sorguları (Parameter Queries) yer alır. En yaygın ve güvenli parametre, kullanıcının kimlik doğrulama belirteci olan JWT'den (JSON Web Token) çıkarılan kullanıcı kimliğidir.

Kullanıcının JWT token'ı PowerSync'e ulaştığında, sistem güvenli bir şekilde token'ın öznesi olan sub bilgisinden kullanıcı ID'sini çeker. YAML dosyasında SELECT request.user_id() as user_id şeklinde tanımlanan bu parametre, daha sonra veri sorgusunda (Data Query) süzgeç olarak kullanılır: SELECT * FROM lists WHERE owner_id = bucket.user_id.

Faz 3: Kullanıcı Kapsamlandırmalı Sync Rules Örneği

edition: 2

bucket_definitions:

user_lists:

Kullanıcı kimliğinin doğrulanmış JWT token üzerinden alınması

parameters:

SELECT request.user_id() as user_id

Yalnızca eşleşen satırların senkronize edilmesi

data:

- SELECT * FROM lists WHERE owner_id = bucket.user_id

Bu mimarinin arka planında devasa bir işlem gücü optimizasyonu yatmaktadır. Eğer sistemde 1, 2 ve 3 ID'li kullanıcılar mevcutsa, PowerSync motoru veritabanındaki değişiklikleri (Postgres WAL üzerinden) okuyarak bu kullanıcılar giriş yapmadan önce bile user_lists["1"], user_lists["2"] ve user_lists["3"] adında sanal kovaları proaktif olarak hazırlayıp güncel tutar. 1 numaralı kullanıcı uygulamayı açtığında, sunucu anında hesaplamalar yapmak yerine zaten hazır bekleyen user_lists["1"] kovasını saniyeden çok daha kısa bir sürede cihaza kopyalar. Bu önden hesaplamalı (preemptive) mimari, uygulamanın anlık reaktif arayüzlere (instantly reactive UI) sahip olmasını sağlarken, aynı zamanda milyonlarca kullanıcıya yatay olarak ölçeklenebilmenin anahtarıdır.

Ayrıca, senkronizasyon verimliliğini artırmak amacıyla Faz 3 mimarisinde "Önceliklendirilmiş Senkronizasyon" (Prioritized Sync) stratejileri devreye alınır. Kritik finansal tabloların cihaz açılır açılmaz senkronize edilmesi gerekirken, sohbet geçmişi veya profil avaturları gibi ikincil veriler daha sonra indirilebilir. Bu durum YAML dosyasında ya doğrudan kova tanımına priority: 1 şeklinde bir anahtar eklenerek, ya da tek bir parametre sorgusu içinde farklı veriler için dinamik olmayan _priority özniteliği seçilerek (örneğin SELECT id AS project_id, 2 AS _priority FROM projects) gerçekleştirilir. Öncelikler tamamen statik olmalı ve parametre sorgusundaki satır

değerlerine bağlı olarak değişkenlik göstermemelidir.

İstemci Tarafı Şema Yönetimi, Drift ORM Entegrasyonu ve Veri Göçü (Migration)

Veritabanı tabanlı projelerin Faz 3 evriminde en sıkıntılı süreçlerden biri, sürekli eklenen yeni özellikler doğrultusunda veritabanı şemalarının (tablolar, sütunlar, ilişkiler) değişime uğraması ve bu değişimin milyonlarca cihazdaki yerel SQLite veritabanlarına kopma yaşanmadan yansıtılmasıdır. Klasik yapılarda sunucu tarafında eklenen yeni bir sütun, istemci uygulamasının eski bir versiyonunu kullanan cihazda anında "şema uyumsuzluğu" (schema mismatch) hatalarına neden olur.

PowerSync mimarisi, bu mühendislik problemini "Şemasız Senkronizasyon" (Schemaless Sync) konseptiyle radikal bir biçimde çözer. PowerSync, arka uçtan indirdiği ham verileri şemasız bir şekilde depolar ve ardından istemci tarafından tanımlanan SQLite görünümüleri (views) aracılığıyla bu veriyi yapılandırır. Bu eşsiz mimari, sunucuda şema değişse dahi PowerSync SDK'sının bu değişiklikleri zarif bir biçimde (gracefully) idare edebilmesini sağlar; böylece istemci tarafında her küçük değişiklik için kod yazılmış açık göçlere (explicit migrations) olan ihtiyaç büyük ölçüde ortadan kalkar.

Flutter ve Dart ekosisteminde PowerSync verilerini yönetmek ve yerel veritabanında kompleks işlemler yapmak için Drift Persistans kütüphanesi (ORM) kullanılmaktadır. Geliştiriciler, drift_sqlite_async paketini kullanarak PowerSync'in eşzamanlama motoruyla Drift'in esnek sorgulama yeteneklerini birleştirirler. Drift ORM, PowerSync tarafında gerçekleşen arka uç değişim bildirimlerini doğrudan yakalayarak Dart akışlarına (streams) iletir. Böylece veritabanında bir satır değiştiği anda, Flutter arayüzündeki (UI) bileşen anında yeniden çizilir ve mükemmel bir reaktif deneyim elde edilir.

Yerel SQLite şemasının doğrudan Drift üzerinden yönetildiği senaryolarda, istemci tarafı şeması tamamen Dart sözdizimi ve .drift dosyaları aracılığıyla tanımlanır ve uygulamanın onCreate geri çağırımı (callback) fonksiyonunda oluşturulur. Şemaya yeni bir sütun eklendiğinde, geliştiricinin MigrationStrategy sınıfı içindeki onUpgrade metodunu dikkatli bir şekilde yapılandırması gerekir. Alternatif olarak Electric SQL gibi mimarilerde şema, Postgres tarafında bir .sql dosyasında tanımlanıp CLI araçlarıyla doğrudan SQLite ifadelerine dönüştürülürken ; PowerSync ve Drift birleşiminde şema otonomisi istemciye aittir. Bu durum geliştiriciye Dart dilinin tip güvenliği (type safety) avantajını sonuna kadar kullanır. Ancak geliştiricinin, Drift tarafında tanımladığı şema ile Supabase tarafında kurduğu Postgres şemasının aynı hizaya geldiğinden sürekli olarak emin olması gerekir.

Çatışma Çözümleme (Conflict Resolution) Stratejileri ve Uç Durumların Yönetimi

Bağımsız çalışan birden fazla istemcinin aynı veri kümesi üzerinde aynı anda çevrimdışı değişiklik yaptığı durumlarda, bağlantı kurulduğunda sunucuda meydana gelen olaylar dizisine "Çatışma" (Conflict) adı verilir. Faz 3 seviyesindeki bir platformda, bir istemci uzun süre çevrimdışı kalıp yeniden bağlandığında, yerel değişikliklerinin diğer istemcilerin o süre zarfında aynı veride yaptığı değişikliklerle mantıksal bir uzlaşmaya (reconciliation) girmesi kesin bir gerekliliktir. Eğer Kullanıcı 1, Kullanıcı 2'den sonra bir işlem gerçekleştirdiyse, sistemdeki tüm istemcilerin aynı olay sıralamasını gözlemlemesi, veri bütünlüğünün en kritik kuralıdır.

PowerSync, geliştiricileri çevrimdışı yetenekleri tamamen kısıtlamak yerine, çatışmalarla nasıl başa çıkacaklarına kendilerinin karar verebileceği bir takım alet çantasıyla donatır. Sistemin çekirdeğinde, sağlanan SupabaseConnector aracılığıyla varsayılan olarak "Son Yazan Kazanır" (Last-Write-Wins - LWW) stratejisi uygulanır. Birçok paylaşımlı durum (shared-state)

senaryosunda, örneğin bir sohbet mesajının düzenlenmesi veya bir sayaç verisinin güncellenmesinde, LWW algoritması mükemmel şekilde çalışır ve ek mühendislik gerektirmez. Veriler istemciden yerel SQLite'a, oradan yükleme kuyruğuna (upload queue), ardından uploadData() metoduyla arka uca (Postgres, MySQL, MongoDB vb.) iletilir. En son zaman damgasına sahip paket arka uçta tutulur.

Ancak, aynı belgenin iki farklı paragrafının aynı anda değiştirilmesi veya bir finansal bilançoda eşzamanlı girdi yapılması gibi gelişmiş iş birliği gerektiren senaryolarda LWW yetersiz kalır. Faz 3 mimarisinde böyle durumlar için Çatışmasız Çoğaltılmış Veri Türleri (Conflict-Free Replicated Data Types - CRDT) entegrasyonu devreye girer. PowerSync'in sağladığı esneklik sayesinde, Yjs gibi güçlü CRDT kütüphaneleri sisteme entegre edilerek eşzamanlı metin veya grafik düzenlemeleri kusursuzca birleştirilebilir. Alternatif olarak, Matthew Weidner'ın yaklaşımında olduğu gibi, tamamen projeye özel karmaşık birleştirme algoritmaları (custom merge algorithm) yazılabilir.

Çatışmaları mimari olarak yönetmenin bir diğer etkili yolu ise, pragmatik kısıtlamalar (Domain-Specific Resolution) getirmektir. Uygulamanın tamamen çevrimdışı çalışmasına izin verilirken, çatışma yaşandığında geri döndürülmesi imkansız olan "yıkıcı işlemlerin" (Destructive Actions) -örneğin bir projenin tamamen silinmesi, hesabın kapatılması veya geri alınamaz bir ÖHVPS bakiye transferinin tetiklenmesi- ağ bağlantısı yokken kullanıcı arayüzünde devre dışı bırakılmasıdır. Uygulamayı tamamen salt okunur (read-only) yapmak yerine, kullanıcıya operasyonel basitlik sunmak adına çevrimdışı iken yıkıcı eylem kapasitesinden feragat edilir. Her durumda yapılan tüm işlemlerin arka planda bir Denetim/Etkinlik günlüğüne (Audit/Activity Log) yazılması ve gerektiğinde PGAudit gibi araçlarla incelenebilmesi, kurumsal Faz 3 uygulamalarında veri yönetişimi (governance) açısından zorunludur.

Standart veri satırlarının aksine, dosya yüklemeleri (PDF dekontlar, kimlik görselleri, profil fotoğrafları) çevrimdışı durumlarda çok daha kırılgan bir yapı sergiler. PowerSync bu durum için "attachments helper" (eklenti yardımcısı) modülünü sunar. Telegram benzeri "gönder-ve-unut" (send-and-forget) katmanı sağlayan bu yardımcı, kullanıcı dosyayı seçtiği an onu yerel depolamada sıraya alır (enqueue), bağlantı geldiğinde arka uca yüklenmesini ve eşzamanlanmasını (syncing) sağlar, ağ hatalarında otonom olarak yeniden dener (retrying) ve işlem başarıyla sonuçlandığında cihazın depolama alanını temizler (cleaning up).

Çatışma Çözümleme Stratejisi	Temel Algoritma ve Çalışma Mantığı	Faz 3 Senaryosu / Kullanım Alanı
Son Yazan Kazanır (LWW)	Cihazların zaman damgasına göre sunucuya ulaşan en güncel işlemin geçerli sayılması.	Profil güncellemeleri, TODO listeleri, sohbet mesajı düzenlemeleri.
CRDT (Çatışmasız Çoğaltım)	Veri mutasyonlarının matematiksel kurallarla (ör: Yjs) hiçbir veri kaybı olmadan kaynaştırılması.	Ortak doküman düzenleme (Google Docs mantığı), kolektif tahta çizimleri.
Yıkıcı İşlemlerin Devre Dışı Bırakılması	Çatışma ihtimalinde geri dönüşü olmayan fonksiyonların çevrimdışıyken arayüzde kapatılması.	Hesap silme, yüksek meblağlı finansal transferler (ÖHVPS), kalıcı veri imhası.
Dosya (Attachment) Yönetimi	Send-and-Forget (Gönder ve Unut). Yerel kuyruğa alma, bağlantı anında yeniden	Açık bankacılık işlem dekontlarının, müşteri kimlik görsellerinin ve raporların

Çatışma Çözümleme Stratejisi	Temel Algoritma ve Çalışma Mantığı	Faz 3 Senaryosu / Kullanım Alanı
	deneme (retry) ve cihazdan temizleme.	buluta yüklenmesi.

Satır Bazlı Güvenlik (RLS), Postgres Yetkilendirmeleri ve Veritabanı Mimarisi

Çevrimdışı yetenekleri ne kadar gelişmiş olursa olsun, Faz 3 ölçeğindeki bir sistemin güvenlik mimarisi "Sıfır Güven" (Zero-Trust) prensibi üzerine inşa edilmelidir. Mimarinin veri tabanı çekirdeğinde Supabase'in gücünü aldığı PostgreSQL yer almaktadır. PowerSync'in istemcilere hangi verilerin "indirileceğini" (download) kontrol eden Senkronizasyon Kuralları (Sync Streams) mükemmel çalışsa da, bu kurallar asla veritabanına veri "yüklenirken" (upload) otorite olarak kabul edilemez. İstemci tarafı her zaman manipülasyona açıktır ve kötü niyetli bir kullanıcı yerel veritabanını değiştirerek arka uca sahte paketler gönderebilir.

İşte bu noktada PostgreSQL'in "Satır Bazlı Güvenliği" (Row Level Security - RLS) devreye girer. RLS, veritabanına ulaşan tüm CRUD operasyonlarına uygulanan en son ve kesin güvenlik duvarıdır. Sync Streams ve RLS her ne kadar birbirini tamamlayıcı (complementary) ve genelde birbirini yansıtan (mirror) teknolojiler olsa da, asıl otoriter set RLS'dir. Örneğin bir kullanıcı, sahibi olmadığı bir listeyi değiştirmeye çalışırsa, RLS politikası veritabanı seviyesinde işlemi reddedecektir. Supabase tarafında kurulması gereken örnek bir RLS kural seti şu şekildedir:

```
-- Kullanıcıların sadece kendilerine ait (sahibi oldukları) listelere erişim politikası
```

```
CREATE POLICY "owned lists" ON public.lists
FOR ALL TO authenticated
USING (auth.uid() = owner_id);
```

```
-- Alt tablolarla (görevlerde), ana tabloya (listeler) bağlı sahipliğin sorgulanması
```

```
CREATE POLICY "todos in owned lists" ON public.todos
FOR ALL TO authenticated
USING (
  auth.uid() IN (
    SELECT lists.owner_id
    FROM public.lists
    WHERE lists.id = todos.list_id
  )
);
```

PowerSync sunucusunun (ister Supabase üzerinde çalışsın, ister self-hosted olsun) PostgreSQL üzerindeki tüm değişiklikleri okuyabilmesi ve cihazlara yayabilmesi için katı RLS kurallarını yasal bir şekilde aşabilen özel yetkili bir veritabanı rolüne ihtiyacı vardır. Geliştirici, PostgreSQL yapılandırmasında PowerSync için replikasyon (REPLICATION) ve RLS'yi atlama (BYPASSRLS) yetkilerine sahip bir kullanıcı (örneğin powersync_role) oluşturmalıdır. ALTER DEFAULT PRIVILEGES komutuyla, gelecekte oluşturulacak tüm şema tabloları için de bu role SELECT yetkisi tanımlanmalıdır. PowerSync sunucusu yapılandırılırken service.yaml veya çevresel değişkenler (environment variables) üzerinden bu kullanıcının kimlik bilgileri ve URI

adresi tanımlanır.

Güvenliği maksimize etmek için, PowerSync ile Supabase (PostgreSQL) arasındaki bağlantının siber saldırılara karşı korunması elzemdir. PowerSync, varsayılan olarak Supabase'in Kök Sertifika Otoritesi (CA) sertifikasını içerdiği için, ek bir yapılandırma gerektirmeden en yüksek güvenlik seviyesi olan verify-full SSL modunu kullanabilir. Sadece yerel test ortamlarında veya dışa kapalı özel ağlarda disable ssl modu kullanılabilir, ancak Faz 3 ticarileşme aşamasında bu kesinlikle kabul edilemez. Alternatif veritabanı kullanımlarında, örneğin MongoDB Atlas için bağlantı kurulduğunda asgari olarak readWrite ve dbAdmin rollerine sahip bir veritabanı kullanıcısı yapılandırılmalıdır.

Edge Bilişim (Edge Computing), Zamanlanmış Görevler (Cron) ve Arka Plan Entegrasyonları

Uygulamanın veri senkronizasyon yetenekleri ne kadar mükemmel olursa olsun, bankacılık entegrasyonu (ÖHVPS), yapay zeka vektör eşleştirmeleri, günlük analiz raporlarının oluşturulması ve üçüncü parti API'lerle haberleşme gibi asenkron ve uzun süren arka plan işlemleri (background jobs) için ayrı bir hesaplama katmanına ihtiyaç vardır. Faz 3 vizyonunda bu süreçler, monolitik sunucular yerine küresel olarak dağıtılmış Deno Edge Fonksiyonları (Deno Edge Functions) aracılığıyla yürütülür.

Supabase'in Deno çalışma zamanı (runtime) tabanlı Edge Fonksiyonları, kodunuzun dünyadaki kullanıcılara en yakın veri merkezlerinde çalışmasını sağlayarak ağ gecikmesini (latency) minimuma indirir. TypeScript destekli bu fonksiyonlar, güçlü tür güvenliğinin (type safety) yanı sıra Deno'nun güvenlik odaklı yapısı ve modern JavaScript API'leri sayesinde inanılmaz hızlı soğuk başlangıçlara (cold start) sahiptir. Ayrıca NPM modülleri ve Node.js yerleşik kütüphaneleriyle tam uyumlu çalışabildiği için mevcut yazılım ekosistemiyle kolayca entegre edilebilir.

Düzenli bankacılık tahsilatları veya haftalık bülten gönderimleri gibi zaman veya olay tetiklemeli operasyonlar için Supabase Cron (Zamanlanmış Görevler) hayati önem taşır. PostgreSQL'in pg_cron ve pg_net eklentilerini (extensions) kullanan bu mimaride, veritabanının kendisi adeta bir görev yöneticisine (scheduler) dönüşür. pg_cron düzenli zaman aralıklarında (örneğin her sabah saat 08:00'de) tetiklenir ve pg_net aracılığıyla belirlenen Edge Fonksiyonuna asenkron bir HTTP POST isteği gönderir.

```
-- pg_cron ve pg_net uzantılarının aktifleştirilmesi
```

```
create extension if not exists pg_net;  
create extension if not exists pg_cron;
```

```
-- Edge fonksiyonunu tetikleyecek arka plan fonksiyonunun oluşturulması
```

```
create or replace function call_notification_processor() returns void  
language sql  
as $$  
    select net.http_post(  
        url :=  
'https://<proje-kodu>.functions.supabase.co/notification-processor',  
        headers := jsonb_build_object('Authorization', 'Bearer  
<servis-rol-anahtari>')  
    );
```

\$\$;

Bu mimari kurulurken, uç fonksiyonların kötü niyetli dış erişimlerden korunması için asla kısıtlı yetkilere sahip "anon" anahtarları (anon key) kullanılmamalı, sistemin tam yetkili "service_role" anahtarları kullanılmalıdır. Ancak bu anahtarlar açık metin halinde SQL koduna gömülme yerine, güvenlik ihlallerini önlemek adına Supabase Vault (kasa) mimarisinde şifrelenerek saklanmalıdır. pg_cron eklentisi zamanlamaları hesaplarken veritabanı sunucusunun zaman dilimini (çoğunlukla UTC) temel aldığından, geliştiricilerin lokal zaman dilimleriyle UTC ofsetlerini (saat farklarını) hesaplayarak tetikleyicileri (trigger) oluşturması hatalı işlem saatlerini önler. Ayrıca pg_net uzantısı üzerinden yapılan HTTP çağrıları "ateşle-ve-unut" (fire-and-forget) felsefesiyle çalışır; yani gönderilen isteğin sonucu doğrudan cron işlemine döndürülemez. Bu asenkron yapı sebebiyle oluşabilecek hataları yakalayabilmek için, veri tabanında özel "Kuyruk Tabloları" (Queue Tables) tasarlanması şarttır. Yazılımın ana entite tabloları (örneğin kullanıcılar, makaleler, finansal işlem kayıtları) temiz bir pattern (tasarım deseni) ile kuyruk tablolarına bağlanır. Edge fonksiyonu, işlemi başarıyla tamamladığında veya hata aldığında bu durumu kuyruk tablosuna işler. Aynı zamanda Sentry gibi gelişmiş hata izleme platformları için Deno çevresel değişkenlerine (Deno.env.get) SENTRY_DSN gibi entegrasyon anahtarları atanarak tüm süreç global çapta monitör edilir. Sistemde e-posta gönderimi, süresi dolmuş verilerin silinmesi (auto-delete), dış mikroservislerin pinglenmesi ve periyodik veri senkronizasyonu gibi tüm ağır yükler, performans kaybı yaşanmaksızın bu uç bilişim ve cron mimarisi üzerinden eritilir.

Ticarileşme Çıktıları ve Faz 3 Başarı Stratejisi

Faz 3 Ar-Ge konseptinin temel misyonu, sınırlı ve laboratuvar ölçeğindeki bir inovasyonun, yüz binlerce kullanıcının aynı anda etkileşime girdiği kaotik gerçek dünya koşullarına (real-world conditions) evrilmesidir. Geliştirilecek bu platform, TÜBİTAK TEYDEB ve 4691 sayılı Kanun'un sağladığı benzersiz vergi istisnaları, ön avans ödemeleri ve hibe rasyoları gibi kaldıraçları kullanarak ar-ge sermayesini çok verimli kullanmak zorundadır. Ticarileşmenin ana damarı olan açık bankacılık ÖHVPS 2.0 entegrasyonu, sistemi sadece bir yazılım olmaktan çıkarıp ulusal finans altyapısına bağlayan kritik bir atılımdır. Günde 12,3 milyon işlemin aktığı bu ekosistemde düzenli ödeme algoritmaları ve kart verisi işleme yetenekleri, işletmeye devasa ve sürdürülebilir bir abonelik nakit akışı yaratacaktır.

Kullanıcı deneyiminin kesintisizliğini temin eden PowerSync destekli Çevrimdışı Öncelikli mimari, Riverpod ve Drift ORM tabanlı Flutter istemcileriyle birleştiğinde, pazarın en reaktif ve hızlı mobil/web deneyimlerinden birini ortaya çıkarır. Geliştirici ekibinin LWW veya CRDT algoritmalarıyla kurguladığı çatışma çözümleri, Supabase tarafındaki kırılmaz RLS güvenlik katmanı ve Deno tabanlı uç bilişim asenkron görev yöneticileri ile desteklendiğinde, sistemin hata toleransı (fault tolerance) maksimuma ulaşır. Faz 3'ün sonunda ürün sadece pazara sunulmuş olmayacak; aynı zamanda global veri gizliliği kurallarına uyan, teknolojik eskimesi öngörülmüş bir patent ömrüne sahip, uluslararası yatırımlara hazır bir teknoloji devrimi olarak ekosistemdeki yerini alacaktır.

Alıntılanan çalışmalar

1. Four Phases of Clinical Trials | Astellas, <https://www.clinicaltrials.astellas.com/four-phases-of-clinical-trials>
2. RAI Explainer: The Drug

Development Process | Perspectives on Innovation - CSIS,
<https://www.csis.org/blogs/perspectives-innovation/rai-explainer-drug-development-process> 3. Clinical Trial Velocity: The Approval Signal Pharma Investors Are Missing,
<https://www.drugpatentwatch.com/blog/clinical-trial-velocity-as-a-signal-for-predicting-drug-approval-and-uptake/> 4. Accelerating the discovery and development of new health technologies | Brookings,
<https://www.brookings.edu/articles/accelerating-discovery-development-new-health-technologies/> 5. TÜBİTAK-TEYDEB Destek Programları Başvuru Ve Yürütme - Etkin Proje,
<https://etkinproje.com/ar-ge-proje-tesvikleri-tubitak-teydeb> 6. Teknoloji ve Yenilik Destek Programları Başkanlığı (TEYDEB) - OAİB,
https://oaib.org.tr/Uploads/haberler_view/6f854c2cc80fbfe37284759d22685f84.pdf 7. 4691 - TEKNOLOJİ GELİŞTİRME BÖLGELERİ KANUNU - Gelir İdaresi Başkanlığı,
<https://www.gib.gov.tr/mevzuat/kanun/476> 8. Teknopark Bilgi Notu - Narter & Partners Hukuk Bürosu, <https://www.narterlaw.com/teknopark-bilgi-notu/> 9. TCMB Açık Bankacılıkta Yeni Fazı Başlattı: ÖHVPS 2.0 Devrede - FT Finansal Teknoloji,
<https://finansalteknoloji.com/haber/tcmb-acik-bankacilikta-yeni-fazi-baslatti-ohvps-2-0-devrede> 10. Açık Bankacılık (Open Banking) Nedir? - Architech, <https://architech.com/kurumsal/blog/teknoloji/acik-bankacilik-ve-turkiyedeki-guncel-durumu/> 11. Bringing Offline-First To Supabase, The Right Way - PowerSync,
<https://powersync.com/blog/bringing-offline-first-to-supabase> 12. Flutter Tutorial: Building An Offline-First Chat App With Supabase And PowerSync,
<https://powersync.com/blog/flutter-tutorial-building-an-offline-first-chat-app-with-supabase-and-powersync> 13. Offline-First Apps Made Simple: Supabase + PowerSync,
<https://powersync.com/blog/offline-first-apps-made-simple-supabase-powersync> 14. Building Offline-First File Uploads with PowerSync Attachments Helper,
<https://powersync.com/blog/building-offline-first-file-uploads-with-powersync-attachments-helper> 15. syncable — Offline-first multi-device sync with Drift and Supabase : r/FlutterDev - Reddit,
https://www.reddit.com/r/FlutterDev/comments/1jr64rh/syncable_offlinefirst_multidevice_sync_with_drift/ 16. Building Local-First Flutter Apps with Riverpod, Drift, and PowerSync | Dinko Marinac,
<https://dinkomarinac.dev/blog/building-local-first-flutter-apps-with-riverpod-drift-and-powersync/> 17. Dart/Flutter ORM Support - PowerSync Docs,
<https://docs.powersync.com/client-sdks/orms/flutter-orm-support> 18. Custom Conflict Resolution - PowerSync Docs, <https://docs.powersync.com/handling-writes/custom-conflict-resolution> 19. Hands-on with React, Supabase, and PowerSync | InfoWorld,
<https://www.infoworld.com/article/4168581/developing-local-first-apps-with-react-supabase-and-powersync.html> 20. Sync Rules From First Principles: Partial Replication To SQLite - PowerSync,
<https://powersync.com/blog/sync-rules-from-first-principles-partial-replication-to-sqlite> 21. Sync Streams and Sync Rules - PowerSync Docs, <https://docs.powersync.com/sync/overview> 22. Sync Rules (Legacy) - PowerSync Docs, <https://docs.powersync.com/sync/rules/overview> 23. Offline-first FlutterFlow apps with PowerSync and Supabase [Guide],
<https://community.flutterflow.io/community-tutorials/post/offline-first-flutterflow-apps-with-powersync-and-supabase-guide-DQyHPTnAycU33ok> 24. Prioritized Sync - PowerSync Docs,
<https://docs.powersync.com/sync/advanced/prioritized-sync> 25. Dart/Flutter SDK - PowerSync Docs, <https://docs.powersync.com/client-sdks/reference/flutter> 26. Backend synchronization - Drift! - Simon Binder, https://drift.simonbinder.eu/examples/server_sync/ 27. Device sync · simolus3 drift · Discussion #2880 - GitHub, <https://github.com/simolus3/drift/discussions/2880>

28. Flutter Drift Database Migration Explained | Fix Schema Changes Without Losing Data, <https://www.youtube.com/watch?v=usKjXq9yaaU> 29. Offline-First Apps with TanStack DB and PowerSync, <https://powersync.com/blog/offline-first-apps-with-tanstack-db-and-powersync> 30. Processing large jobs with Edge Functions, Cron, and Queues - Supabase, <https://supabase.com/blog/processing-large-jobs-with-edge-functions> 31. RLS and Sync Streams - PowerSync Docs, <https://docs.powersync.com/integrations/supabase/rls-and-sync-streams> 32. Supabase - PowerSync Docs, <https://docs.powersync.com/integrations/supabase/guide> 33. Self-Hosted Instance Configuration - PowerSync Docs, <https://docs.powersync.com/configuration/powersync-service/self-hosted-instances> 34. PowerSync Setup Guide, <https://docs.powersync.com/intro/setup-guide> 35. Deno Edge Functions | Supabase Features, <https://supabase.com/features/deno-edge-functions> 36. Supabase Cron | Schedule Recurring Jobs in Postgres, <https://supabase.com/modules/cron> 37. Supabase Cron, <https://supabase.com/blog/supabase-cron> 38. How to Set Up Cron Jobs with Supabase Edge Functions Using pg_cron | by Samuelmpwanyi | Medium, <https://medium.com/@samuelmpwanyi/how-to-set-up-cron-jobs-with-supabase-edge-functions-using-pg-cron-a0689da81362>